

Web Data → Campaign Automation

Working Guide — Demo Project

A step-by-step guide for setting up, running, verifying, and presenting the Python web-data campaign automation demonstration.

Component	Technology
Language	Python
Web scraping	Requests + BeautifulSoup
Data processing	Native Python lists/dictionaries
Database	Supabase / PostgreSQL
Campaign platform	Mock advertising platform
Testing	Pytest
Operating environment	Windows 11 + VS Code / CMD

1. Project Objective

This demo demonstrates an end-to-end workflow that collects information from permitted public web pages, cleans and deduplicates the data, enriches it with simple industry classification, assigns an audience segment, stores the results in Supabase, creates a mock advertising campaign, and calculates campaign metrics.

Pipeline:

```
graph TD
    A[Allowed Public Websites] --> B[Python Scraper]
    B --> C[Data Cleaning]
    C --> D[Deduplication]
    D --> E[Enrichment]
    E --> F[Audience Segmentation]
    F --> G[Supabase]
    G --> H[Mock Campaign Generator]
    H --> I[Mock Campaign Metrics]
```

2. Prerequisites

- Windows 11 with Python installed.
- VS Code or another code editor.
- Internet access for the demo websites and Supabase.
- A Supabase account and project.
- The project source code available locally.

Note: This demonstration does not require a local PostgreSQL installation and does not use a real advertising API or spend real money.

3. Project Structure

```
web-data-campaign-automation/
├──
├── app/
│   ├── __init__.py
│   ├── pipeline.py
│   ├── scraper/
│   │   ├── website_scraper.py
│   │   ├── processing/
│   │   │   ├── cleaner.py
│   │   │   ├── deduplicator.py
│   │   │   ├── enrichment/
│   │   │   │   ├── enricher.py
│   │   │   │   ├── segmentation/
│   │   │   │   │   ├── segmenter.py
│   │   │   │   │   ├── database/
│   │   │   │   │   │   ├── supabase_client.py
│   │   │   │   │   │   ├── campaign/
│   │   │   │   │   │   │   ├── mock_platform.py
│   │   │   │   │   │   │   ├── analytics/
│   │   │   │   │   │   │   │   ├── metrics.py
│   │   │   │   │   │   │   │   ├──
│   │   │   │   │   │   │   │   ├── data/
│   │   │   │   │   │   │   │   │   ├── raw/
│   │   │   │   │   │   │   │   │   ├── scripts/
│   │   │   │   │   │   │   │   │   │   ├── run_pipeline.py
│   │   │   │   │   │   │   │   │   │   ├── tests/
│   │   │   │   │   │   │   │   │   │   │   ├── test_cleaner.py
│   │   │   │   │   │   │   │   │   │   │   ├── test_segmenter.py
│   │   │   │   │   │   │   │   │   │   │   ├── .env
│   │   │   │   │   │   │   │   │   │   │   ├── .env.example
│   │   │   │   │   │   │   │   │   │   │   ├── .gitignore
│   │   │   │   │   │   │   │   │   │   │   ├── requirements.txt
```

4. Open the Project in VS Code

Open Command Prompt and move into the project directory:

```
cd /d F:\web-data-campaign-automation
code .
```

If VS Code is already open, simply open the project folder **F:\web-data-campaign-automation**.

5. Create and Activate the Virtual Environment

In the VS Code terminal using CMD:

```
cd /d F:\web-data-campaign-automation
python -m venv venv
venv\Scripts\activate
```

The terminal should show **(venv)** at the beginning of the command prompt.

6. Install Dependencies

The project intentionally avoids Pandas because the Windows environment encountered an Application Control policy issue with Pandas DLLs.

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

requirements.txt

```
requests
beautifulsoup4
supabase
python-dotenv
pytest
lxml
```

7. Configure Supabase

Create a Supabase project. The Python application uses Supabase as the hosted PostgreSQL database.

Create the required tables in Supabase → SQL Editor → New Query.

```
CREATE TABLE scraped_pages (
  id BIGSERIAL PRIMARY KEY,
  url TEXT UNIQUE NOT NULL,
  title TEXT,
  description TEXT,
  industry TEXT,
  segment TEXT,
  scraped_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE campaigns (
  id BIGSERIAL PRIMARY KEY,
  external_campaign_id TEXT,
  name TEXT NOT NULL,
  platform TEXT,
  objective TEXT,
  segment TEXT,
  headline TEXT,
  description TEXT,
  cta TEXT,
  status TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE TABLE campaign_metrics (
  id BIGSERIAL PRIMARY KEY,
  external_campaign_id TEXT,
  impressions INTEGER DEFAULT 0,
  clicks INTEGER DEFAULT 0,
  conversions INTEGER DEFAULT 0,
  spend NUMERIC DEFAULT 0,
  ctr NUMERIC,
  cpc NUMERIC,
  conversion_rate NUMERIC,
  recorded_at TIMESTAMPTZ DEFAULT NOW()
);
```

For this demo, enable the temporary RLS policies below:

```
CREATE POLICY "Allow insert scraped pages"
ON public.scraped_pages
FOR INSERT TO anon
WITH CHECK (true);
```

```
CREATE POLICY "Allow read scraped pages"
ON public.scraped_pages
FOR SELECT TO anon
USING (true);
```

```
CREATE POLICY "Allow insert campaigns"
ON public.campaigns
FOR INSERT TO anon
WITH CHECK (true);
```

```
CREATE POLICY "Allow read campaigns"
ON public.campaigns
FOR SELECT TO anon
```

```

USING (true);

CREATE POLICY "Allow insert campaign metrics"
ON public.campaign_metrics
FOR INSERT TO anon
WITH CHECK (true);

CREATE POLICY "Allow read campaign metrics"
ON public.campaign_metrics
FOR SELECT TO anon
USING (true);

```

Demo security note: These broad anon policies are suitable only for this temporary demonstration. A production system should use authentication and appropriately restricted database policies/service credentials.

8. Configure the .env File

Create a file named `.env` in the project root:

```

SUPABASE_URL=https://YOUR_PROJECT_ID.supabase.co
SUPABASE_KEY=YOUR_SUPABASE_KEY
SCRAPER_USER_AGENT=Mozilla/5.0

```

Replace the placeholders with the Supabase Project URL and API key from the Supabase project settings. Never commit `.env` to GitHub.

9. Verify the Supabase Connection

Create or run a small connection test such as:

```

from app.database.supabase_client import supabase

response = (
    supabase
    .table("scraped_pages")
    .select("*")
    .limit(1)
    .execute()
)

print("Supabase connection successful!")
print(response.data)

```

Run it with:

```
python test_supabase.py
```

Expected result: **Supabase connection successful!** followed by an empty list or existing rows.

10. Run the Automated Tests

```
pytest -q
```

The completed demo currently reports **4 passed**. Supabase client deprecation warnings may appear; they do not indicate test failure.

11. Run the Full Pipeline

Use module execution from the project root:

```
python -m scripts.run_pipeline
```

Important: Use `python -m scripts.run_pipeline` rather than `python scripts/run_pipeline.py`, because the module form correctly resolves the `app` package.

12. Expected Successful Output

```
=====
WEB DATA → CAMPAIGN AUTOMATION PIPELINE
=====

[1/7] Starting website scraping...
[SCRAPER] Scraping: https://example.com
[SCRAPER] Scraping: https://example.org
[SCRAPER] Collected 2 records.

[2/7] Cleaning data...
[DATA] Records after cleaning: 2

[3/7] Removing duplicates...
[DATA] Records after deduplication: 2

[4/7] Enriching and segmenting data...

[5/7] Saving data to Supabase...
[DATABASE] Scraped data saved successfully.

[6/7] Creating campaigns...
[CAMPAIGN] Created: B2B Technology Campaign

[7/7] Collecting campaign metrics...
[METRICS] B2B Technology Campaign
  Impressions      : 10000
  Clicks           : 450
  Conversions      : 35
  Spend            : $180.0
  CTR              : 4.5%
  CPC              : $0.4
  Conversion Rate   : 7.78%

=====
PIPELINE COMPLETED SUCCESSFULLY
=====
```

13. Verify the Results in Supabase

Open Supabase → Table Editor and check these tables:

- **scraped_pages** — should contain the scraped URLs, titles, descriptions, industry, and segment.
- **campaigns** — should contain the generated mock campaign.
- **campaign_metrics** — should contain impressions, clicks, conversions, spend, CTR, CPC, and conversion rate.

For this demo, the expected campaign metrics are:

Metric	Expected value
Impressions	10,000
Clicks	450
Conversions	35
Spend	\$180.00
CTR	4.50%
CPC	\$0.40
Conversion rate	7.78%

14. How the Pipeline Works

1. **Scraping** — Requests downloads the public HTML and BeautifulSoup extracts the page title and meta description.
2. **Cleaning** — URLs, titles, and descriptions are normalized and whitespace is cleaned.
3. **Deduplication** — Duplicate URLs are removed.
4. **Enrichment** — Simple keyword rules classify records into an industry.
5. **Segmentation** — Industry is mapped to an audience segment such as B2B Technology.
6. **Database** — Processed records are stored in Supabase.
7. **Campaign** — A mock platform creates campaign metadata without contacting a real ad platform.
8. **Analytics** — Mock impressions, clicks, conversions, and spend are used to calculate CTR, CPC, and conversion rate.

15. Responsible Demo Usage

- Use only public websites and data you are permitted to collect.
- Do not bypass authentication, CAPTCHAs, robots restrictions, rate limits, or anti-bot controls.
- Avoid collecting unnecessary personal or sensitive information.
- Use reasonable request timeouts and request rates.
- The current campaign platform is intentionally mocked and does not launch real advertising campaigns.
- Keep Supabase credentials in .env and never commit them.

16. Troubleshooting

ModuleNotFoundError: No module named 'app'

Run the pipeline from the project root using:

```
python -m scripts.run_pipeline
```

RLS error: new row violates row-level security policy

Confirm that the temporary INSERT policies in Section 7 were created in Supabase SQL Editor.

Supabase connection error

Check SUPABASE_URL and SUPABASE_KEY in .env, then rerun the connection test.

Pandas DLL/Application Control error

Pandas is intentionally not part of this demo. The current implementation uses native Python data structures.

17. Final Demo Checklist

- Virtual environment activated.
- Dependencies installed.
- Supabase tables created.
- Demo RLS policies created.
- .env configured and excluded from Git.
- Supabase connection verified.

- `pytest` passes.
- `python -m scripts.run_pipeline` completes successfully.
- `scraped_pages`, `campaigns`, and `campaign_metrics` verified in Supabase.

Demo status: End-to-end pipeline successfully tested locally with Supabase. The `example.com` and `example.org` URLs are technical placeholders for demonstration purposes.

Prepared as a practical working guide for the Web Data → Campaign Automation demo.